Practical 4

Strings, StringBuilder & Regular Expressions

Part A — Practice Programs (Hour 1)

1) Password Strength — four regex rules

Code

PasswordChecker.java

```
public class PasswordChecker {
    static boolean hasLength(String pw)  { return pw.length() >= 8; }
    static boolean hasUpper(String pw)   { return pw.matches(".*[A-Z].*"); }
    static boolean hasDigit(String pw)   { return pw.matches(".*[0-9].*"); }
    static boolean hasSpecial(String pw) { return pw.matches(".*[^A-Za-z0-9].*"); }

    static int score(String pw) {
        int n = 0;
        if (hasLength(pw))  n++;
        if (hasUpper(pw))   n++;
        if (hasDigit(pw))   n++;
        if (hasSpecial(pw)) n++;
        return n;
    }

    static String strength(String pw) {
        int n = score(pw);
        if (n <= 1) return "Weak";
        if (n <= 3) return "Medium";
        return "Strong";
    }
}
```

Driver.java

```
public class Driver {
    public static void main(String[] args) {
        String[] tests = { "abc", "abcdefgh", "Abcdefgh", "Abcd1234", "Abcd1234!" };
        for (String pw : tests) {
            System.out.println("Password: " + pw);
            System.out.println("  length>=8: " + PasswordChecker.hasLength(pw)
                    + ", upper: " + PasswordChecker.hasUpper(pw)
                    + ", digit: " + PasswordChecker.hasDigit(pw)
                    + ", special: " + PasswordChecker.hasSpecial(pw));
            System.out.println("  Result: " + PasswordChecker.strength(pw));
        }
    }
}
```

Output

```
Password: abc
  length>=8: false, upper: false, digit: false, special: false
  Result: Weak
Password: abcdefgh
  length>=8: true, upper: false, digit: false, special: false
  Result: Weak
Password: Abcdefgh
  length>=8: true, upper: true, digit: false, special: false
  Result: Medium
Password: Abcd1234
  length>=8: true, upper: true, digit: true, special: false
  Result: Medium
Password: Abcd1234!
  length>=8: true, upper: true, digit: true, special: true
  Result: Strong
```

2) Chat-Log Filter — split(" ", 3) and StringBuilder

Code

ChatFilter.java

```
public class ChatFilter {
    static String[] split(String line) {
        return line.split(" ", 3);
    }

    static boolean matches(String message, String keyword) {
        return message.toLowerCase().contains(keyword.toLowerCase());
    }
}
```

Driver.java

```
import java.util.Scanner;

public class Driver {
    public static void main(String[] args) {
        String[] logs = {
            "10:05 alice Hello there",
            "10:06 bob",
            "10:07 carol See you at lunch"
        };
        Scanner s = new Scanner(System.in);
        System.out.print("Enter keyword: ");
        String keyword = s.nextLine();

        int count = 0;
        StringBuilder report = new StringBuilder();
        for (String line : logs) {
            String[] parts = ChatFilter.split(line);
            if (parts.length < 3) {
                System.out.println("Skipped malformed line: \"" + line + "\"");
                continue;
            }
            if (ChatFilter.matches(parts[2], keyword)) {
                count++;
                report.append(parts[0]).append(" ").append(parts[1]).append(":
                    ").append(parts[2]).append("\n");
            }
        }
        System.out.println("Matches: " + count);
        System.out.print(report);
        s.close();
    }
}
```

Output

```
Enter keyword: hello
Skipped malformed line: "10:06 bob"
Matches: 1
10:05 alice: Hello there
```

The keyword hello is typed by the user at the prompt; the malformed line "10:06 bob" has fewer than 3 parts and is skipped.

3) Template Filler (For Advanced Learners — home) — Pattern and Matcher

Code

TemplateFiller.java

```
import java.util.regex.*;

public class TemplateFiller {
    static String fill(String template, String[] names, String[] values) {
        Pattern p = Pattern.compile("\\{(\\w+)\\}");
        Matcher m = p.matcher(template);
        StringBuilder out = new StringBuilder();
        int last = 0;
        while (m.find()) {
            out.append(template, last, m.start());
            String key = m.group(1);
            String value = "[?]";
            for (int i = 0; i < names.length; i++) {
                if (names[i].equals(key)) { value = values[i]; break; }
            }
            out.append(value);
            last = m.end();
        }
        out.append(template.substring(last));
        return out.toString();
    }
}
```

Driver.java

```
public class Driver {
    public static void main(String[] args) {
        String template = "Dear {name}, order {id} ships {date}.";
        String[] names  = { "name", "id" };
        String[] values = { "Riya", "A07" };
        System.out.println("Template: " + template);
        System.out.println("Filled  : " + TemplateFiller.fill(template, names, values));
    }
}
```

Output

```
Template: Dear {name}, order {id} ships {date}.
Filled  : Dear Riya, order A07 ships [?].
```

Part B — Project Work (Hour 2): MiniBank

MiniBank must never trust raw input. This lab adds a validation layer (regex checks for mobile, email, PAN, IFSC), a command parser that turns "DEPOSIT AC0001 500" into a Command object, and a StringBuilder statement. Account and Customer are unchanged from Practical 3.

1) Validator — compiled Pattern objects

Code

```
import java.util.regex.Pattern;

public class Validator {
    private static final Pattern MOBILE = Pattern.compile("^[6-9][0-9]{9}$");
    private static final Pattern EMAIL  =
        Pattern.compile("^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$");
    private static final Pattern PAN    = Pattern.compile("^[A-Z]{5}[0-9]{4}[A-Z]$");
    private static final Pattern IFSC   = Pattern.compile("^[A-Z]{4}0[A-Z0-9]{6}$");
    private static final Pattern AMOUNT = Pattern.compile("^[1-9][0-9]*$");

    public static boolean isValidMobile(String s) { return s != null &&
        MOBILE.matcher(s).matches(); }
    public static boolean isValidEmail(String s)  { return s != null && EMAIL.matcher(s).matches();
        }
    public static boolean isValidPan(String s)    { return s != null && PAN.matcher(s).matches(); }
    public static boolean isValidIfsc(String s)   { return s != null && IFSC.matcher(s).matches(); }
    public static boolean isValidAmount(String s) { return s != null &&
        AMOUNT.matcher(s).matches(); }
}
```

Supplementary problem covered: isValidAmount(String) accepts only positive whole numbers.

2) TransactionType enum and Command record

Code

TransactionType.java

```
public enum TransactionType {
    DEPOSIT, WITHDRAW, TRANSFER
}
```

Command.java

```
public record Command(TransactionType type, String accountNumber, long amount) { }
```

3) CommandParser — split() and conversion

Code

```
public class CommandParser {
    public static Command parse(String line) {
        String[] parts = line.trim().split("\\s+");
        if (parts.length != 3) {
            throw new IllegalArgumentException("Expected 3 parts (TYPE ACCOUNT AMOUNT) but got " +
                parts.length);
        }
        TransactionType type = TransactionType.valueOf(parts[0].toUpperCase());
        long amount = Long.parseLong(parts[2]);
        return new Command(type, parts[1], amount);
    }
}
```

Supplementary problem covered: parse() reports an error when the line has the wrong number of parts.

4) StatementFormatter — StringBuilder

Code

```
public class StatementFormatter {
    public static String buildStatement(Account account) {
        StringBuilder sb = new StringBuilder();
        sb.append("======== MiniBank Statement ========\n");
        sb.append("Account Number : ").append(account.getAccountNumber()).append("\n");
        sb.append("Owner          : ").append(account.getOwnerName()).append("\n");
        sb.append("Balance        : ").append(account.getBalance()).append("\n");
        sb.append("Status         : ").append(account.isActive() ? "Active" :
            "Closed").append("\n");
        sb.append("====================================");
        return sb.toString();
    }
}
```

5) main — test each validator, parse a command, print a statement

Code

```
public class MiniBank {
    public static void main(String[] args) {
        System.out.println("--- Validators ---");
        System.out.println("isValidMobile(\"9876543210\") = " +
            Validator.isValidMobile("9876543210"));
        System.out.println("isValidMobile(\"12345\")      = " + Validator.isValidMobile("12345"));
        System.out.println("isValidEmail(\"abc@xyz.com\") = " +
            Validator.isValidEmail("abc@xyz.com"));
        System.out.println("isValidEmail(\"abc@xyz\")     = " + Validator.isValidEmail("abc@xyz"));
        System.out.println("isValidPan(\"ABCDE1234F\")    = " + Validator.isValidPan("ABCDE1234F"));
        System.out.println("isValidPan(\"ABCD12345\")     = " + Validator.isValidPan("ABCD12345"));
        System.out.println("isValidIfsc(\"SBIN0001234\")  = " +
            Validator.isValidIfsc("SBIN0001234"));
        System.out.println("isValidIfsc(\"SBIN1001234\")  = " +
            Validator.isValidIfsc("SBIN1001234"));
        System.out.println("isValidAmount(\"500\")        = " + Validator.isValidAmount("500"));
        System.out.println("isValidAmount(\"-5\")         = " + Validator.isValidAmount("-5"));

        System.out.println("--- Command parser ---");
        Command cmd = CommandParser.parse("DEPOSIT AC0001 500");
        System.out.println("type=" + cmd.type() + ", accountNumber=" + cmd.accountNumber() + ",
            amount=" + cmd.amount());
        try {
            CommandParser.parse("DEPOSIT AC0001");
        } catch (IllegalArgumentException e) {
            System.out.println("Error: " + e.getMessage());
        }

        System.out.println("--- Statement ---");
        Account a = new Account("Riya", 4000);
        System.out.println(StatementFormatter.buildStatement(a));
    }
}
```

Output

```
--- Validators ---
isValidMobile("9876543210") = true
isValidMobile("12345")      = false
isValidEmail("abc@xyz.com") = true
isValidEmail("abc@xyz")     = false
isValidPan("ABCDE1234F")    = true
isValidPan("ABCD12345")     = false
isValidIfsc("SBIN0001234")  = true
isValidIfsc("SBIN1001234")  = false
isValidAmount("500")        = true
isValidAmount("-5")         = false
--- Command parser ---
type=DEPOSIT, accountNumber=AC0001, amount=500
Error: Expected 3 parts (TYPE ACCOUNT AMOUNT) but got 2
--- Statement ---
======== MiniBank Statement ========
Account Number : AC0001
Owner          : Riya
Balance        : 4000
Status         : Active
====================================
```

Package layout, build and tag

Code

```
minibank/
 |-- Account.java
 |-- Command.java
 |-- CommandParser.java
 |-- Customer.java
 |-- MiniBank.java
 |-- StatementFormatter.java
 |-- TransactionType.java
 +-- Validator.java

javac *.java
java MiniBank
cd ..
git add .
git commit -m "Practical 4 - validation and command parsing"
git tag lab-04
```

Every validator returns true for valid input and false for invalid input, and parse() returns the correct Command (type=DEPOSIT, accountNumber=AC0001, amount=500).

Key Questions / Analysis / Interpretation

1. What is the difference between String, StringBuilder and StringBuffer, and when is each preferred?

String is immutable: every change creates a new object, which is safe and fine for constants and small text. StringBuilder is a mutable buffer and is the fastest choice when building text in loops inside one thread (such as the statement). StringBuffer is the older synchronized (thread-safe) version of StringBuilder; it is slower and only needed when several threads append to the same buffer.

2. What does the regular expression [6-9][0-9]{9} match, and why use anchors?

It matches exactly ten digits whose first digit is 6, 7, 8 or 9 (an Indian mobile number). Without anchors, matcher.find() would also succeed on a longer string that merely contains such a number, for example 'call 9876543210 now' or a 12-digit number. Anchors ^ and $ (or matches()) force the whole input to match the pattern.

3. Why should input be validated at the boundary before it is used?

Everything typed by a user or read from a file may be wrong or malicious. Checking it once, where it enters the system, keeps bad data out of the model, gives a clear error message at the right place, and stops problems such as wrong account numbers or negative amounts from spreading through the rest of the program.